

Sonu Kumar

Data Analyst and Aspiring Data Scientist



SQL



Power BI



Excel



Python

Pizza Sales Data Analysis

Leveraging SQL to transform raw point-of-sale data into actionable business intelligence.



Map temporal sales trends and daily operational demands.



Identify best- and worst-performing products by volume and revenue.

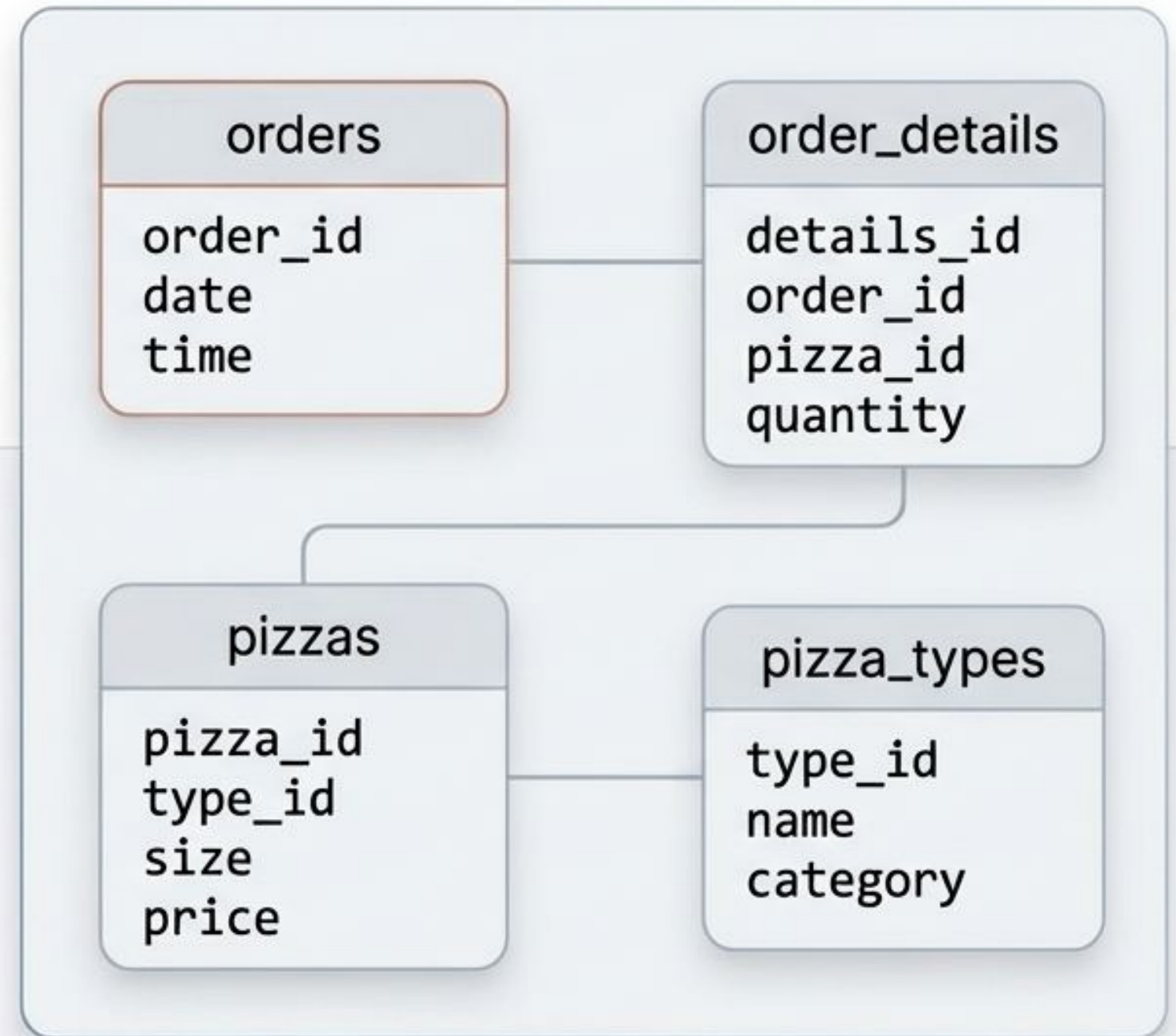


Uncover deep revenue drivers and category contributions.



Data Architecture & Schema setup

```
1  create database pizzahut;  
2  
3  create table orders (  
4    order_id int not null,  
5    order_date date not null,  
6    order_time time not null,  
7    primary key(order_id)  
8  );  
9  
10 create table order_details (  
11   order_details_id int not null,  
12   order_id int not null,  
13   pizza_id text not null,  
14   quantity int not null,  
15   primary key(order_details_id)  
16 );
```



Core KPIs: Establishing Baseline Volume and Value

Volume

21,350 Total Orders

```
select count(order_id) as  
total_orders  
from orders;
```

total_orders
21350

Value

\$817,860 Total Revenue

```
select round(sum(pizzas.price *  
order_details.quantity),2) as  
total_revenue  
from order_details  
join pizzas  
on order_details.pizza_id =  
pizzas.pizza_id;
```

total_revenue
817860.05



The business successfully processed over 21,000 individual orders, generating nearly \$818k in topline revenue during the analyzed period.

Product Preferences: Premium Pricing and Dominant Sizes

Highest-Priced Pizza

```
1  -- Identify the highest-priced pizza.
2
3  • select pizza_types.name, pizzas.price from
4    pizza_types join pizzas on
5    pizza_types.pizza_type_id = pizzas.pizza_type_id order by
6    pizzas.price desc limit 1;
```

Result Grid			Filter Rows:		Export:		Wrap Cell Content:		Fetch rows:	
	name	price								
	The Gresk Pizza	35.95								

 The Greek Pizza holds the top price point at \$35.95.

Most Common Pizza Size

```
1  -- Identify the most common pizza size ordered.
2
3  * select pizzas.size, count(order_details.order_details_id) as order_count from
4  pizzas join order_details on
5  pizzas.pizza_id = order_details.pizza_id group by
6  pizzas.size order by order_count desc;
```

Result Grid   Filter Rows: Expert:  View Call Content: 

	ISS	order_point
▶	L	10395
	M	10325
	S	11137
	M	544
	XXL	28

 Large (L) is overwhelmingly the most popular size with **18,526** units sold.

! Customer preference skews heavily toward the "Large" format, which makes up the vast majority of all volume.

Inventory management should prioritize Large-sized ingredients and boxes.

Top Sellers by Pure Order Volume

The screenshot shows a database management interface. On the left, a 'SCHEMAS' pane lists databases like 'newcdrema' and 'piczakut', with 'piczakut' expanded to show tables like 'order_details', 'orders', 'pizzas_types', and 'pizzas'. The main area displays a SQL query to find the top 5 most ordered pizza types. Below the query, a 'Result Grid' shows the results of the query, listing the pizza names and their quantities. At the bottom, a 'Table: pizzas' section lists columns like 'pizza_id', 'pizza_type_id', 'size', and 'price'. The bottom right shows an 'Action Output' pane with a log of database actions.

```
-- List the top 5 most ordered pizza types
select pizza_types.name, sum(order_details.quantity) as quantity
from pizza_types join pizzas on
pizza_types.pizza_type_id = pizzas.pizza_type_id
order_details on
order_details.pizza_id = pizzas.pizza_id
group by pizza_types.name
order by quantity desc
limit 5
```

name	quantity
The Classic Deluxe Pizza	2453
The Barbecue Chicken Pizza	2432
The Hawaiian Pizza	2422
The Pepperoni Pizza	2418
The Thai Chicken Pizza	2371

Table: pizzas

Columns:

- pizza_id: text
- pizza_type_id: text
- size: text
- price: double

Result 4 x

Output

Action Output

#	Time	Action
108	16.05:19	select round(sum(pizzas price * order_details.quantity),2) as total_revenue from ord
109	16.05:37	select round(sum(pizzas price * order_details.quantity),2) as total_revenue from ord

1

The Classic Deluxe Pizza
(2,453)

2

The Barbecue Chicken Pizza
(2,432)

3

The Hawaiian Pizza
(2,422)

The 'Classic Deluxe' and 'Barbecue Chicken' pizzas lead the menu in raw sales volume. The top three are separated by less than 40 units, indicating strong, distributed demand across familiar profiles.

Category Dynamics: Menu Offerings vs. Actual Volume

Menu Offerings

```
1 -- Join relevant tables to find the category-wise distribution of pizzas.
2
3 • select category, count(name) from
4   pizza_types group by
5   category;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

category	count(name)
Chicken	6
Classic	8
Supreme	9
Veggie	9

Supreme (9)

Veggie (9)

Classic (8)

Chicken (6)

Sales Volume

```
1 -- Join the necessary tables to find the total quantity
2 -- of each pizza category ordered.
3
4 • select pizza_types.category, sum(order_details.quantity) as quantity from
5   pizza_types join pizzas on
6   pizza_types.pizza_type_id = pizzas.pizza_type_id join
7   order_details on
8   order_details.pizza_id = pizzas.pizza_id group by
9   pizza_types.category order by quantity desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

category	quantity
Classic	14888
Supreme	11987
Veggie	11649
Chicken	11050

Classic (14,888)

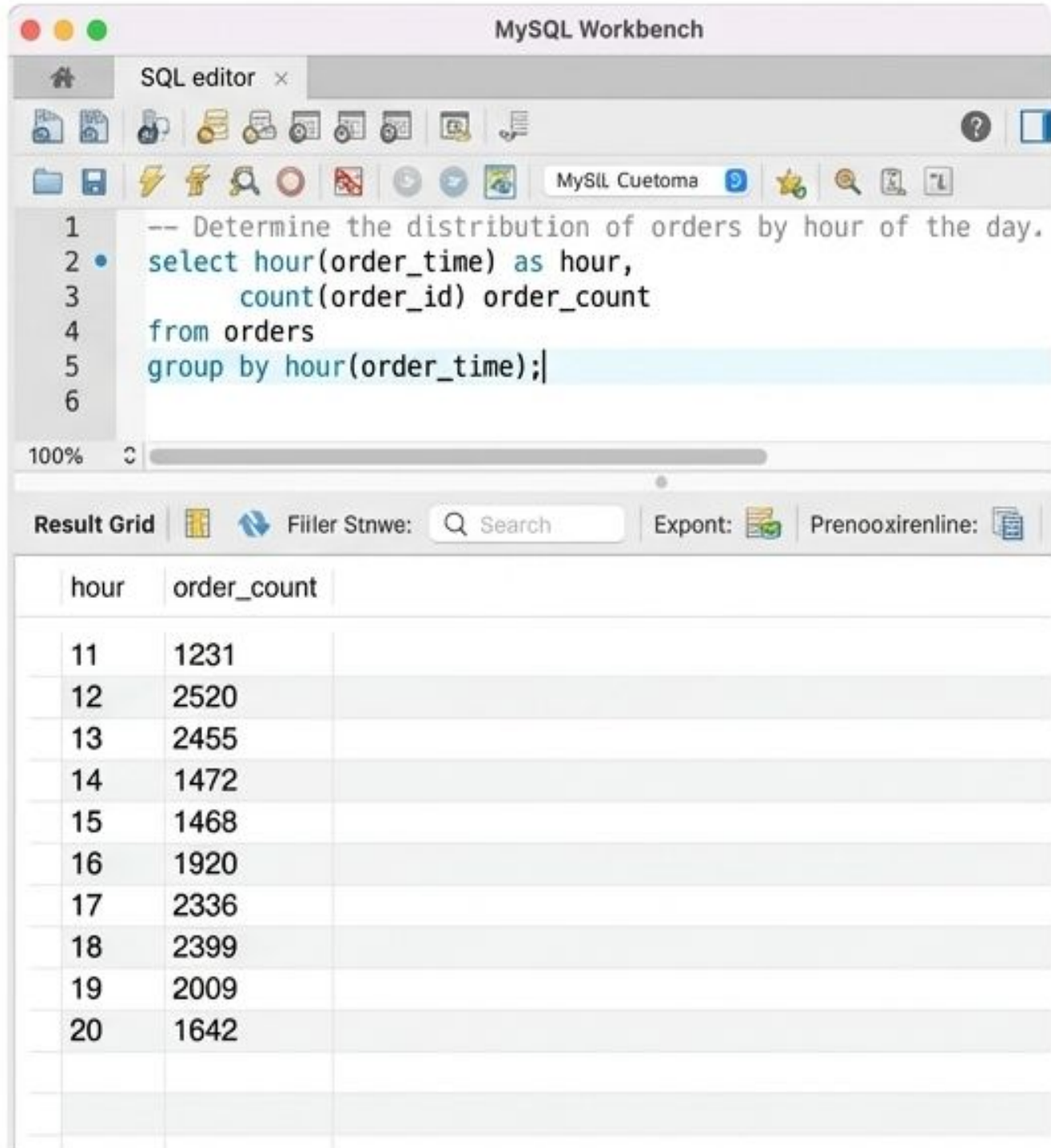
Supreme (11,987)

Veggie (11,649)

Chicken (11,050)

A fascinating operational mismatch: While 'Supreme' and 'Veggie' offer the most menu variety, the 'Classic' category dominates total volume. Customers prefer familiar classics despite having fewer options to choose from.

Temporal Demand: Mapping the Daily Rushes



! Clear twin-peak demand. Staffing and kitchen prep must be heavily optimized for the 12-1 PM and 5-7 PM windows, while mid-afternoon lulls offer time for restocking.

Daily Operational Load (Subquery Aggregation)

```
-- Group the orders by date and calculate the average number of
pizzas ordered per day.
select round(avg(quantity_sum),0) as avrg_pizza_ordered_per_day
from
  (select orders.order_date,
    sum(order_details.quantity) as quantity_sum
  from orders
  join order_details
    on orders.order_id = order_details.order_id
  group by orders.order_date) as order_qty;
```

Utilized a nested subquery to first calculate individual daily totals, then applied an outer aggregation to find the baseline average.

138

Average Pizzas Per Day

📈 On average, the kitchen needs to be prepped to produce 138 pizzas daily. This forms the baseline metric for daily dough and ingredient preparation.

Revenue Drivers: Shifting from Volume to Value

SQL Query & Output

```
1  -- Determine the top 3 most ordered pizza types based on revenue
2
3 • select pizza_types.name, sum(order_details.quantity * pizzas
4     pizza_types join pizzas on
5     pizza_types.pizza_type_id = pizzas.pizza_type_id join
6     order_details on
7     pizzas.pizza_id = order_details.pizza_id group by
8     pizza_types.name order by revenue desc limit 3;
```

Result Grid	Filter Rows:	Experts	Wrap Call Content:	Fetch rows:
name	REVENUE			
The Thai Chicken Pizza	43434.25			
The Barbecue Chicken Pizza	42758			
The California Chicken Pizza	41409.5			

The Financial Reality

1. Thai Chicken

\$43,434

2. Barbecue Chicken

\$42,768

3. California Chicken

\$41,409



Despite 'Classic Deluxe' leading in raw unit volume, the 'Thai Chicken' pizza claims the #1 spot for gross revenue. High-value specialty pizzas are the true financial engines of the menu.

Category Revenue Contribution (Advanced Subqueries)

SQL Query & Calculation

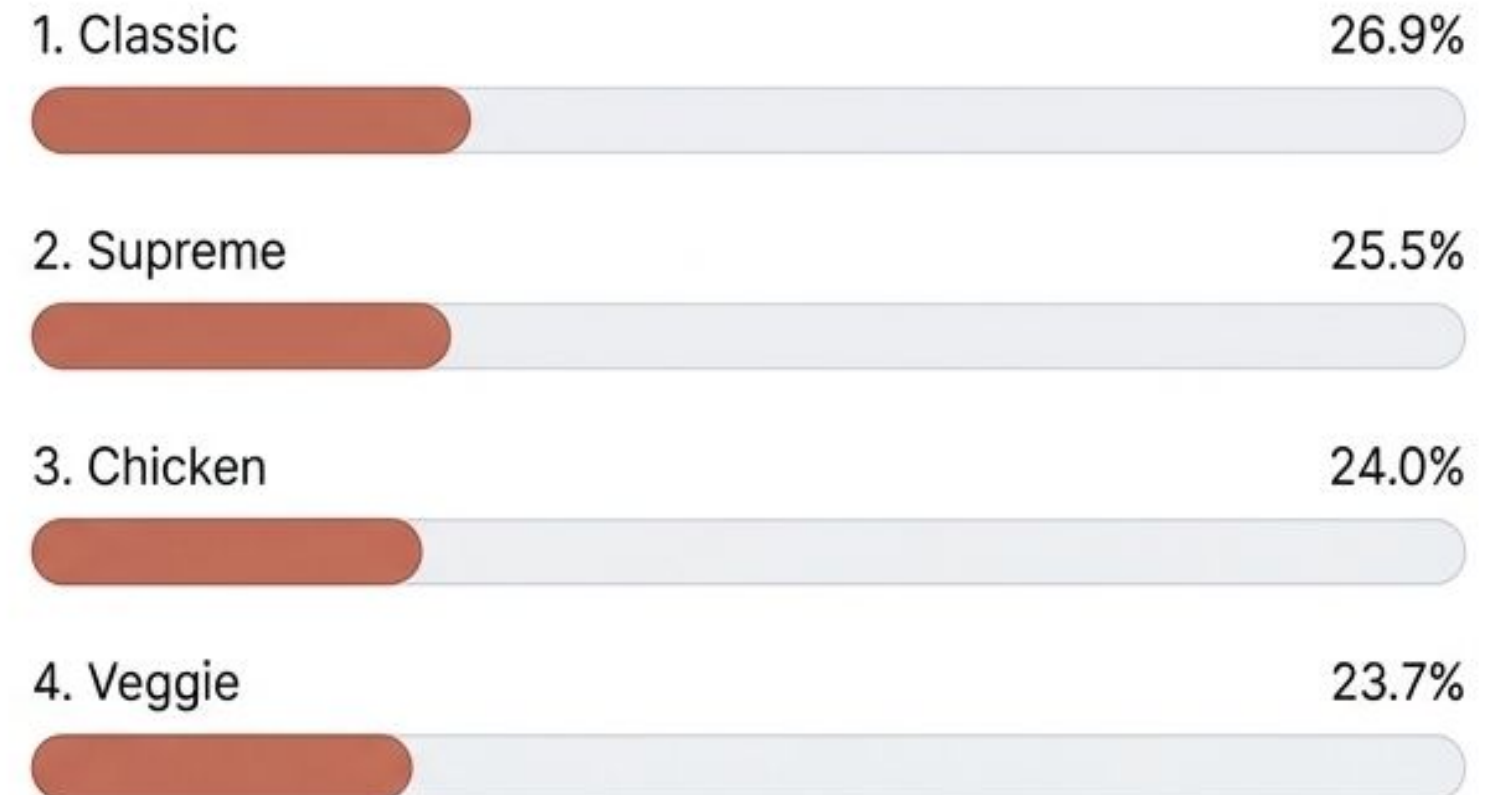
```
MySQL Workbench

-- Calculate the percentage contribution of each pizza type to total revenue.
select pizza_types.category,
       round((sum(order_details.quantity * pizzas.price) /
              (select round(sum(pizzas.price * order_details.quantity),2) as total
               from order_details
               join pizzas on order_details.pizza_id = pizzas.pizza_id))
              * 100, 2) as revenue_percentage
from pizza_types
join pizzas
  on pizza_types.pizza_type_id = pizzas.pizza_id
join order_details
  on pizzas.pizza_id = order_details.pizza_id
group by pizza_types.category
order by revenue_percentage desc;
```

Complex subquery serves as the total revenue denominator for percentage calculation.

Utilized a complex subquery within the main query to determine the overall total revenue, which then serves as the denominator for calculating each category's percentage contribution.

Category Balance



An incredibly balanced menu. No single category monopolizes revenue; all four sit comfortably between 23% and 27%. This indicates a healthy, diversified product catalog that minimizes reliance on a single demographic.

Tracking Cumulative Growth with Window Functions

```
1  -- Analyze the cumulative revenue generated over time.
2
3  • select order_date,
4     sum(revenue) over(order by order_date) as cum_revenue from
5     (select orders.order_date, sum(order_details.quantity * pizzas.p
6     revenue from
7     order_details join pizzas on
8     order_details.pizza_id = pizzas.pizza_id join
9     orders on
10    orders.order_id = order_details.order_id group by
11    orders.order_date) as sales;
```

order_date	cum_revenue
2015-01-01	2713.8500000000004
2015-01-02	5445.75
2015-01-03	8158.25
2015-01-04	9663.6
2015-01-05	11929.55
2015-01-06	14358.5
2015-01-07	16563.7
2015-01-08	19299.65

Leveraged Window Functions to create a running total of sales. This allows us to track cumulative financial growth day-by-day without grouping away the temporal data.

By January 8th alone, the business had **cumulatively generated over \$19,399**, demonstrating strong, consistent daily cash flow at the start of the year.

Isolating Category Champions (CTEs & Ranking)

```
-- Determine the top 3 most ordered pizza types based on revenue for each pizza category.  
WITH ( @pizza_category) {  
  SELECT category, top, category  
    JOIN category in pizzaalona_=> pizza.specific  
    JOIN category = in.revenue  
    GROUP BY category, revenue  
} CTE ( revenue) {  
  WHERE category = rank() over(partition by category order by revenue desc)  
}  
FROM top, category ;
```

Employed a Common Table Expression (CTE) combined with a Ranking Window Function to partition the data by category, cleanly isolating the top 3 revenue performers within each distinct menu segment.

This query empowers regional managers to identify the 'anchor' products for **localized marketing campaigns** within any specific category.

Executive Insights & Operational Strategy

Product & Marketing Strategy

- Focus inventory heavily on Large sizes.
- Heavily market the Thai Chicken (top revenue) and Classic Deluxe (top volume) as the menu anchors.

Operations & Staffing

- Optimize kitchen staffing exclusively around the twin peaks: 12-1 PM and 5-7 PM.
- Set daily ingredient prep baselines to accommodate an average of 138 pizzas.

Menu Optimization

- Maintain current category ratios. The revenue split (23-27% per category) shows perfect balance.
- Consider trimming unique options in Supreme/Veggie since Classic drives more volume with fewer options.

Thank You.

Always exploring data, currently open for new analytical opportunities and constructive feedback.



sonu786786



/in/sonu-kumar51/



View the full repository
and raw queries online.